Jared Benman

ITMM-444

11-02-2026

Weather Advisor Lab Questions

NWS Calls

Two upstream calls are needed because the NWS API only takes latitude and longitude, so you look up the zip code to get the coordinates.

Step 1

One of the success lines is the Chicago zip code, 60616. Since the cache output a miss, it didn't come from the database and took 301 seconds.

Step 3

A short cache matters more because it allows multiple requests for one zip code without going over the NWS rate limit.

CACHE_SECONDS shouldn't be long because weather changes so frequently and usually should be updated after every hour, so having it too long can lead to inaccurate information.

Step 5

An attack who takes over the advisor tier would be able to see the gemini-api-key as it shows allowed but won't be able to access the db-password as its access is denied.

Step 6

When a security group rule names another security group as its source, it takes account for the resources in the load balancer's security group even if there are IP addresses that change, which a CIDR range can't do.

Step 7

The 30 second pause exists so that the service doesn't keep trying to access the database after it fails, and instead it gets an answer without having to wait for. Without the pause, every request would be waiting on the database even though it's down.

Step 8

The main issue with having an error that only shows up in the log line is that problems would go unnoticed and if something is broken then it would just continue to be broken as no one would be aware of that error.

Step 9

The weather service recovered after 0 seconds. Also, state should live outside the instances because instances in an Auto Scaling Group can be terminated and replaced at any time so then information on the instances would also get deleted.

Why the secrets are in SSM and each is read by one role, instead of in .env or the user-data.

Putting the secrets in SSM keeps them encrypted and away from my files. The app reads them when it runs, using its instance role. A .env file can easily end up on GitHub, and anyone who can view the launch template can read the user-data. Each role can read only one secret, so if the advisor got hacked, the attacker could get the Gemini key but not the database password.

Turning backups off: what it saved, what it cost you, and when you would never do it.

Turning backups off meant no backup storage charges, and there were no snapshots left over that could keep costing money after I deleted the database. This was successfully seen with retention at 0 and empty lists for snapshots and automated backups. The tradeoff for that would be the fact that the data could no longer be restored. Because of that, I would not use this method for important information that needs to be reused or stored anywhere, as I would rather create a backup for that information.

What data you sent to Gemini and why the free tier makes that matter.

The data I sent to Gemini was the zip code that provided the city, state, and forecast, excluding any personal information about anyone using it. This is important on the free tier as it's unsafe to provide that type of data that you're not sure will remain completely private.

Review Questions

Where else could each leak from (shell history, process lists, logs, screenshots, Git)?

- The gemini key could leak from various sources like the shell history if it's not hidden, Git if it was placed into the .env, or even a screenshot. The database password could also be leaked when creating the database instance.

What would change if both roles had ssm:* on *?

- Either tier could read both secrets, plus every other parameter in the account. It could also overwrite or delete them, so someone on the advisor could wipe the DB password.

Why is the ALB open to the internet but the instances and the database are not?

- The ALB is open to the internet because it serves as the entryway while the instances and database are not needed to be accessed by any users. If they were accessed by users, it increases unnecessary threat potential.

What can an advisor instance do that it should not be able to do? How would you fix it?

- The database allows the whole app security group, so the advisor can connect to port 5432 even though it doesn't need to. This can be fixed by giving each tier its own security group and only let the weather group into the database.

What does the certificate check protect against that TLS without it does not?

- TLS encrypts the connection so without the certificate check, it poses risk as it doesn't confirm who you're talking to on the opposite side.

What would you lose if this database held real data, and what would you configure instead?

- If the database got deleted or corrupted, everything would be gone for good, with no restore and no point-in-time recovery. For real data I would keep at least a few days of backups and turn on deletion protection.

Who decides when stored data may be destroyed in a real organization?

- In a real organization, this decision would most likely be decided by the owner of the data as it's their own information being handled.

What could an attacker gain if the page printed raw model output?

- If the page printed raw model output, an attacker could steal cookies or redirect people to a phishing page.

What would you have to change before using a paid or school-owned Gemini account for real users?

- Before using a paid or school account for real user, I would probably take the measures to ensure user's data privacy and limit the amount of access is available.

How could someone use up your quota, and what could you add (rate limiting, a WAF, authentication)?

- Someone could use up my quota by repeatedly running a script that consumes the daily limits in Gemini. In order to prevent that, adding an AWS WAF rate-limit would be very beneficial along with some sort of user authentication.

Why is it important that the response says source: fallback or cache: unavailable?

- It's important that the source says each as clear labels tell the status of the service and prevents silent failures by making it obvious that a backend component is down.